A day without new knowledge is a lost day.

Database Technologies - MySQL

In this module we are going to learn SQL, PL/SQL and NoSQL(MongoDB)

- `sudo apt install build-essential`

MySQL is case-insensitive

Case Sensitivity

Case Sensitivity in Table Names: By default, MySQL's case sensitivity for table names depends on the operating system. On Linux, table names are case-sensitive, whereas on Windows, they are case-insensitive.

Case Sensitivity in Column Names: Column names in MySQL are case-insensitive by default.

Case Sensitivity in Data: By default, string comparisons are case-insensitive because MySQL uses the utf8_general_ci collation (Unicode Transformation Format where "ci" stands for case-insensitive).

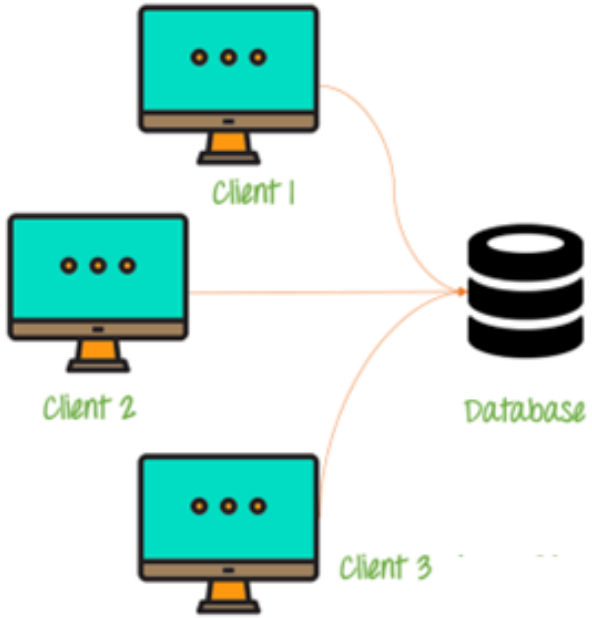
If A and a, B and b, ... are treated in the same way then it is case-insensitive.
MySQL is case-insensitive

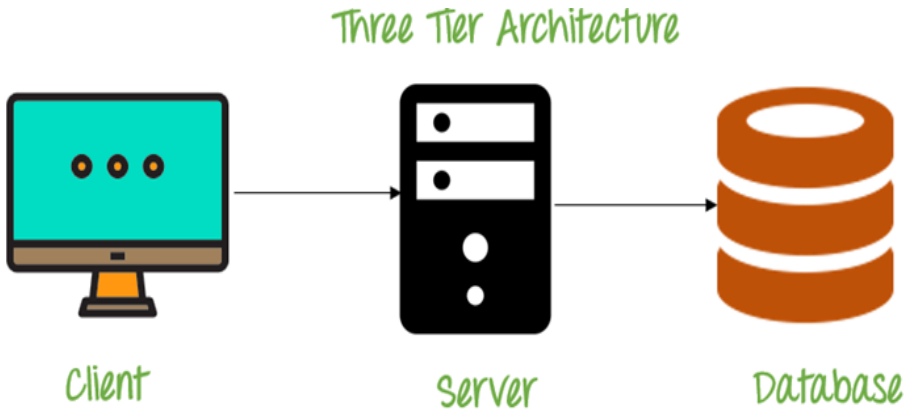
Introduction

- If anyone who wants to develop a good application
  then he should have the knowledge three major components.

They are . . . . .

- Presentation Layer [UI]
- Application Layer [Server Application and Client Application]
- Data Layer [Data Access Object (DAO) / Data Access Layer (DAL)] { Flat Files | RDBMS | NoSQL }





Introduction

Why do we need databases (Use Case)?

We **need databases** because they organize data in a manner which allows us to store, query, sort, and manipulate data in various ways. Databases allow us to do all this things.

What is Relation and Relationship?

Reference / Referential key

Remember:

- A ***reference*** is a relationship between two tables where the values in one table refer to the values in another table. This is usually enforced using a ***foreign key*** constraint to maintain referential integrity.
  a) The referencing column is called the Foreign Key.
  b) The referenced column is usually the Primary Key of the parent table.

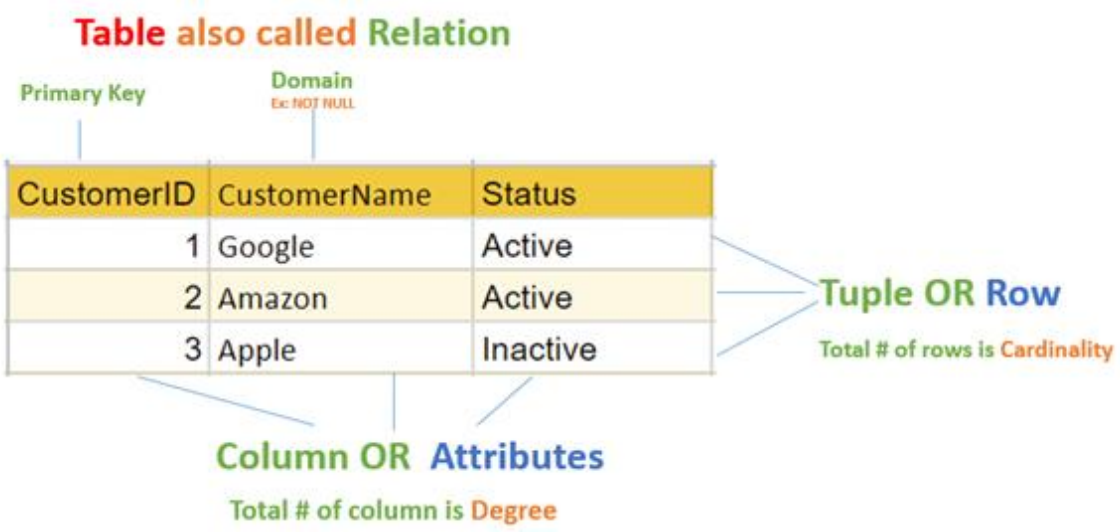
- A ***referential key*** is a column or set of columns in a table that refers to the ***primary key*** of another table. It establishes a relationship between two tables, where one table is called the parent table, and the other is called the child table.

relation and relationship?

Relation ***(in Relational Algebra "R" stands for relation)*****:** In Database, a relation represents a **table** or an **entity** than contain attributes. In Relational Algebra, a relation is a table with rows and columns, just like in a Relational Database Management System (RDBMS). It represents a set of tuples (records) that share the same structure. Relation is a Logical Instantiation/Model of a TABLE.

Relationship: In database, relationship is that how the two entities are **connected** to each other, i.e. what kind of ***relationship type*** they hold between them.

Primary/Foreign key is used to specify this relationship.



Remember:

Foreign Key is also known as

- referential constraint
- referential integrity constraint.

Note:

- **Table -** The physical instantiation of a relation in the database schema.
- **Relation -** A logical construct that organizes data into rows and columns.

File Systems is the traditional way to keep your data organized.

File System
vs
DBMS

A File-Oriented System is the traditional way of storing and managing data before databases (like MySQL, Oracle, . . .)

file-oriented system

File Anomalies

```
struct Employee {
   int emp_no;
   char emp_name[50];
   int salary;
} emp[1000];
```

```
struct Employee {
   int emp_no;
   char emp_name[50];
   int salary;
};
struct Employee emp[1000];
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
.
.
500 sam 3500
.
.
1000 amit 2300
.
.
2000 jerry 4500
.
.
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
.
500  sam 3500
.
3   rajan 4500
.
500  sam 3500
.
.
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
.
sam 500 3500
.
ram  550 5000
.
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
500 sam 3500
.
600 neel 4500
```

file-oriented system

File Anomalies

- Create/Open an existing file
- Reading from file
- Writing to a file
- Closing a file

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300
```

file attributes

- File Name
- Type
- Location

file permissions

- File permissions
- Share permissions

search empl ID=1

```
1 suraj 4000
2 ramesh 6000
3 rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300
```

search emp_name

```
1 suraj 4000
2 ramesh 6000
3 rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300
```

advantages & disadvantage of file-oriented system

advantages of file-oriented system

The biggest advantage of file-based storage is as follows.

- **Backup**: It is possible to take faster and automatic back-up of database stored in files of computer-based systems.
- **Data retrieval:** It is possible to retrieve data stored in files in easy and efficient way.
- **Flexibility:** File systems provide flexibility in storing various types of data, including text documents, images, audio, video, and more
- **Cost-Effectiveness**: File systems often do not incur licensing costs, making them cost-effective for basic data storage needs.
- **Editing**: It is easy to edit any information stored in computers in form of files.
- **Remote access**: It is possible to access data from remote location.
- **Sharing**: The files stored in systems can be shared among multiple users at a same time.

disadvantage of file-oriented system

The biggest disadvantage of file-based storage is as follows.

- **Data redundancy***:* It is possible that the same information may be duplicated in different files. This leads to data redundancy results in memory wastage.
  (Suppose a customer having both kind of accounts - **saving** and **current** account. In such a situation a customer detail are stored in both the file, saving.txt- file and current.txt- file , which leads to Data Redundancy.)
- **Data inconsistency***:* Because of data redundancy, it is possible that data may not be in consistent state.
  (Suppose customer changed his/her address. There might be a possibility that address is changed in only one file (saving.txt) and other (current.txt) remain unchanged.)
- **Limited data sharing***:* Data are scattered in various files and also different files may have different formats (for example: .txt, .csv, .tsv and .xml) and these files may be stored in different folders so, due to this it is difficult to share data among different applications also if the saving account department wants to share data with loan department, they need to manually copy files, leading to delays because File Systems do not support multi-user environments.
- **Data Isolation:** Because data are scattered in various files, and files may be in different formats (for example: .txt, .csv, .tsv and .xml), writing new application programs to retrieve the appropriate data is difficult.
- (Suppose a loan data is in one file and account holder data in another, there is no easy way to analyze account holder data with his loan status.)
- **Data security:** Data should be secured from unauthorized access, for example a account holder in a bank should not be able to see the account details of another account holder, such kind of security constraints are difficult to apply in file processing systems.

disadvantage of file-oriented system

The biggest disadvantage of file-based storage is as follows.

Disadvantage of File-oriented system

- **Data Integrity**: Data integrity refers to the accuracy and consistency of data. In a file-oriented system, enforcing data integrity is difficult because there are no built-in mechanisms to ensure that data is valid or consistent across multiple files.
  (the balance field value must be grater than 5000.)
- **Concurrency Issues:** When multiple users or applications try to access and modify a file at the same time, concurrency problems can arise.
  (if two users attempt to update the same file simultaneously, it can lead to data corruption or loss of data.)
- **Lack of Flexibility:** Modifying the structure of files, such as adding new fields or changing data formats, can be difficult and time-consuming. Changes might require manual updates to each file or even rewriting entire applications that interact with the files.

***Relation Schema*:** A relation schema represents name of the relation with its attributes, every attribute would have an associated domain.

e.g.

- **Student**(rollNo:INT, name:VARCHAR(20), address:VARCHAR(50), phone:VARCHAR(12), age:INT, PRIMARY KEY(rollNo)) is relation schema for STUDENT
- **Customers**(CustomerID:INT, Name:VARCHAR(50), Email:VARCHAR(100), City:VARCHAR(50), PRIMARY KEY(CustomerID)) is relation schema for CUSTOMERS

DBMS

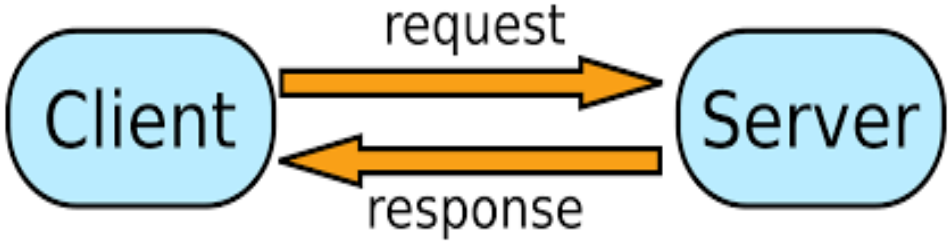
- **database:** Is the collection of **related data** which is **organized,** database can store and retrieve large amount of data easily, which is stored in one or more data files by one or more users, it is called as **structured data.**
- **management system**: it is a software, designed to **define**, **manipulate**, **retrieve** and **manage** data in a database.

relational database management system?

A RDBMS is a database management system (DBMS) that is based on the **relational model** introduced by Edgar Frank Codd at IBM in 1970.

RDBMS supports

- *client/server Technology*
- *Highly Secured*
- *Relationship (PK/FK)*



object relational database management system?

An object database is a database management system in which information is represented in the form of objects.

PostgreSQL is the most popular pure ORDBMS. Some popular databases including Microsoft SQL Server, Oracle, and IBM DB2 also support objects and can be considered as ORDBMS.

Advantage of ORDBMS

- Function/Procedure overloading.
- Extending server functionality with external functions written in C or Java.
- User defined data types.
- Inheritance of tables under other tables.

relational model concepts
and
properties of relational table

properties of relational table

ID	job	firstName	DoB	salary
1	manager	Saleel Bagde	yyyy-mm-dd	••••••
3	salesman	Sharmin	yyyy-mm-dd	••••••
4	accountant	Vrushali	yyyy-mm-dd	ABC
2	salesman	Ruhan	yyyy-mm-dd	••••••
5	9500	manager	yyyy-mm-dd	••••••
5	Salesman	Rahul Patil	yyyy-mm-dd	••••••

Properties of Relational Tables:

- Values are atomic (no multivalued cells).
- Column values are of the same kind. (***Attribute Domain***: Every attribute has some pre-defined datatypes, format, constraints of a column, and defines the range of values that are valid for that column known as attribute domain.)
- Each row is unique.
- The sequence/order of columns is irrelevant – (unimportant).
- The sequence/order of rows is irrelevant – (unimportant).
- Each table name, attribute/column must have a unique name.
- Attributes may contain **NULL** (unknown/missing values)

What is data?

what is data?

Data is any facts that can be stored and that can be processed by a computer.

Data can be in the form of Text or Multimedia

e.g.

- number, characters, or symbol
- images, audio, video, or signal

What is Entity Relationship Diagram?

Entity Relationship Diagram (ER Diagram)

Use E-R model to get a high-level graphical view to describe the **"ENTITIES"** and their **"RELATIONSHIP"**

The basic constructs/components of ER Model are **Entity**, **Attributes** and **Relationships**.

An entity can be a **real-world object.**

What is Entity?

An entity in DBMS is a real-world object that has certain properties called attributes that define the nature of the entity.

entity

In relation to a database , an entity is a

- Person(student, teacher, employee, client, department, …)
- Place(classroom, building, …) --a particular position or area
- Thing(computer, lab equipment, …) --an object that is not named (represents a tangible object)
- Concept(course, batch, student's attendance, …) -- an idea,

about which data can be stored. All these entities have some **attributes** or **properties** that give them their **identity**.

Every entity has its own characteristics.

In Entity Relationship(ER) Model attributes can be classified into the following types.

- Simple/Atomic and Composite Attribute
- Single Valued and Multi Valued attribute
- Stored and Derived Attributes
- Complex Attribute

Remember:

In SQL, the same name can be used for two (or more) attributes as long as the attributes are in different relations.

attributes

- **Simple / Atomic Attribute** --VS-- **Composite Attribute**
  (Can't be divided further) (Can be divided further)
- **Single Value Attribute** --VS-- **Multi Valued Attribute**
  (Only One value) (Multiple values)
- **Stored Attribute** --VS-- **Derived Attribute**
  (Only One value) (Virtual)
- **Complex Attribute**
  (Composite & Multivalued)

Employee ID: An employee ID can be a composite attribute, which is composed of sub-attributes such as department code, job code, and employee number.

- **Atomic Attribute:** An attribute that cannot be divided into smaller independent attribute is known as atomic attribute.
  e.g. ID's, PRN, age, gender, zip, marital status cannot further divide.
- **Single Value Attribute:** An attribute that holds exactly one value for a given record at any point in time is known as single valued attribute. Single-valued attributes are typically used to provide a unique identifier for a record.
  e.g. manufactured part can have only one serial number, voter card ID, blood group, branchID can have only one value.
- **Stored Attribute:** The stored attribute are such attributes which are already stored in the database and from which the value of another attribute is derived.
  e.g. (HRA, DA…) can be derive from salary, age can be derived from DoB, total marks or average marks of a student can be derived from marks, TotalPrice can be derived from Quantity × UnitPrice.

Composite **VS** Multi Valued Attribute

Composite Attribute

composite / multi valued attributes

Person Entity

- *Name* attribute: (`firstName` + `middleName` + `lastName`)
- *PhoneNumber* attribute: (`countryCode` + `cityCode` + `phoneNumber`)
- *Date* attribute: (`Day` + `Month` + `Year`)
- *productDimensions* attribute: (`Length` + `Width` + `Height`)
- *carRegistration* attribute: (`State_Code` + `Series` + `Number`)

{Address}

{street, city, state, postal-code}

{street-number, street-name, apartment-number}

Multi Valued Attribute

Person Entity

- *Hobbies* attribute: [reading, hiking, hockey, skiing, photography, . . .]
- *SpokenLanguages* attribute: [Hindi, Marathi, Gujarati, English, . . .]
- *Degrees* attribute: [10th, 12th, BE, ME, PhD, . . .]
- *emailID* attribute: [saleel@gmail.com, salil@yahoomail.com, . . .]
- *Skills* attribute: [MySQL, Oracle, Redis, MongoDB, Java, . . .]

types of Keys?

Keys are used to establish relationships between tables and also to uniquely identify any record in the table.

types of Keys?

r = Employee(EmployeeID, FullName, job, salary, PAN, DateOfBirth, emailID, deptno)

- **Candidate Key:** are individual columns in a table that qualifies for uniqueness of all the rows. Here in Employee table EmployeeID, PAN or emailID are Candidate keys.
- **Primary Key**: is the columns you choose to maintain uniqueness in a table. Here in Employee table you can choose either EmployeeID, PAN or emailID columns, EmployeeID is preferable choice.
- **Alternate Key**: Candidate column other the primary key column, like if EmployeeID is primary key then , PAN or emailID columns would be the Alternate key.
- **Super Key**: If you add any other column to a primary key then it become a super key, like EmployeeID + FullName or EmployeeID + deptno is a Super Key.
- **Composite Key**: If a table do not have any single column that qualifies for a Candidate key, then you have to select 2 or more columns to make a row unique. Like if there is no EmployeeID, PAN or emailID columns, then you can make FullName + DateOfBirth as Composite key. But still there can be a narrow chance of duplicate row. Ensures data uniqueness in many-to-many relationships. e.g. in order_details table we can have multiple products OrderID + ProductID

What is a Prime, Non-Prime Attribute?

Prime attribute (*Entity integrity*):- An attribute, which is a **part of the prime-key** (candidate key), is known as a prime attribute.

Consider a relation Student(StudentID, Name, Email, Phone).

- ***Candidate Keys:*** {StudentID}, {Email}, {Phone}
- ***Prime Attributes:*** StudentID, Email, Phone (since they are part of a Candidate Key).

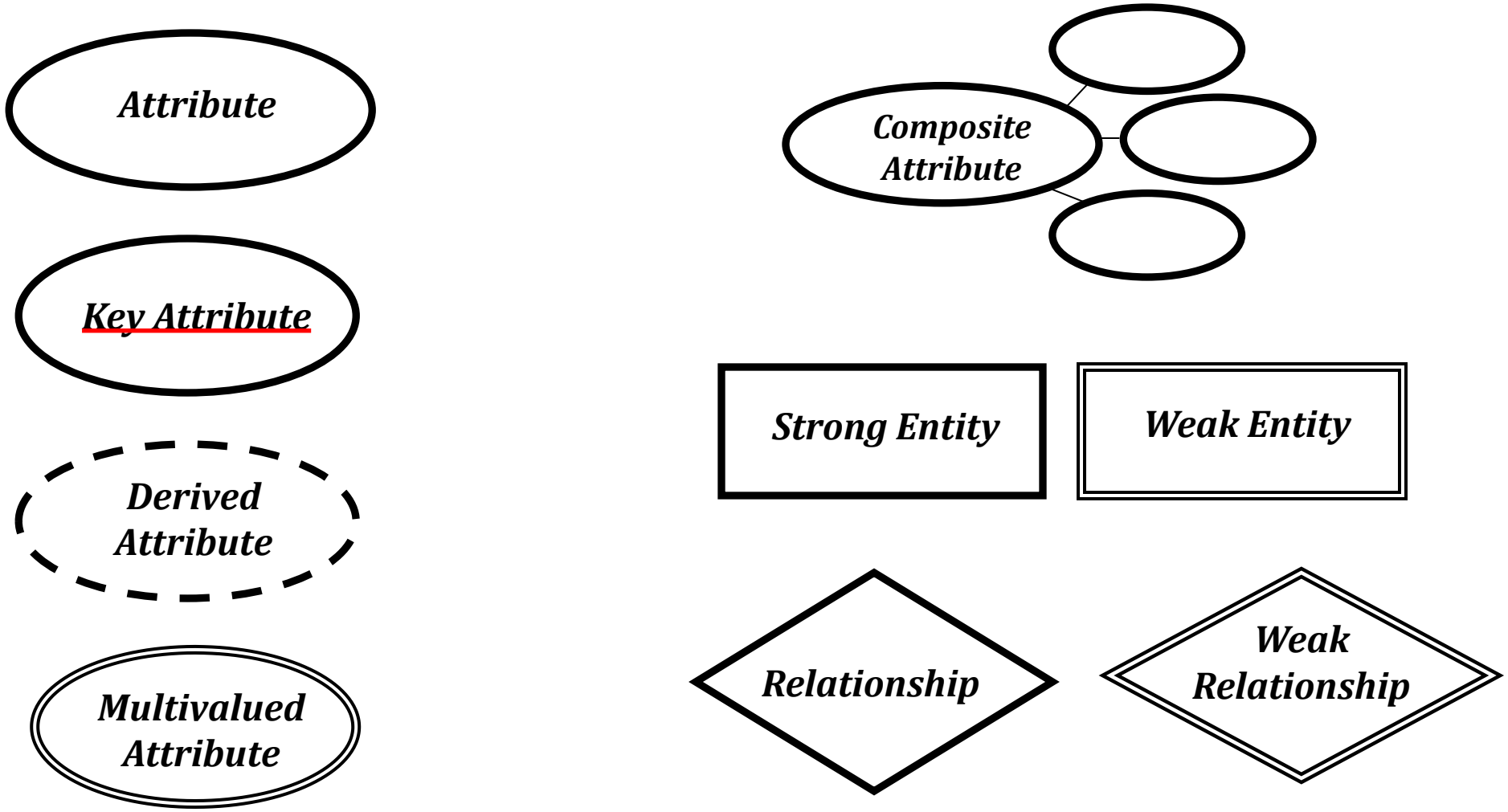
Non-prime attribute:- An attribute, which is **not a part of the prime-key** (candidate key), is said to be a non-prime attribute.

In the Student(StudentID, Name, Email, Phone) relation:

- ***Candidate Keys:*** {StudentID}, {Email}, {Phone}
- ***Prime Attributes:*** StudentID, Email, Phone
- ***Non-Prime Attribute:*** Name (because it is not part of any Candidate Key).

Entity Relationship Diagram Symbols

entity relationship diagram symbols



strong and weak entity

Strong Entity: A strong entity is not dependent on any other entity in the schema. A strong entity will always have a primary key. Strong entities are represented by a single rectangle.

Weak Entity: A weak entity is dependent on a strong entity to ensure its existence. Unlike a strong entity, a weak entity does not have any primary key. A weak entity is represented by a double rectangle. The relation between one strong and one weak entity is represented by a double diamond. This relationship is also known as ***identifying relationship***.

What is a degree, cardinality and union in database?

What is a degree, cardinality and union in database?

- **Degree d(R) / Arity**: Total number of **attributes/columns** present in a relation/table is called **degree of the relation** and is denoted by $\mathbf{d}(R)$.

- **Cardinality |R|**: Total number of **tuples/rows** present in a relation/table, **is called cardinality of a relation** and is denoted by $|R|$, which changes dynamically as rows are inserted or deleted.

  Cardinality is the numerical relationship between rows of one table and rows in another. Common cardinalities include *one-to-one*, *one-to-many*, and *many-to-many*.

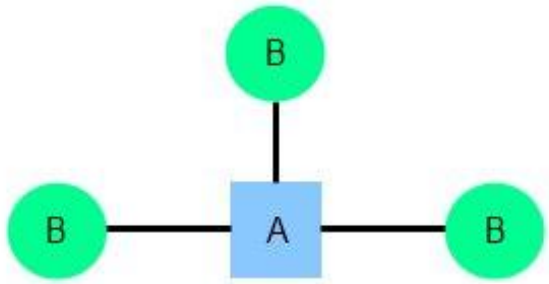
- **Union Compatibility**: Two relations ***R*** and ***S*** are set to be Union Compatible to each other if and only if:
  1. They have the **same degree** $\mathbf{d(R)}$.
  2. Domains of the respective attributes should also be same.
  3. Column names don't need to match, but positions must match.

Common relationships

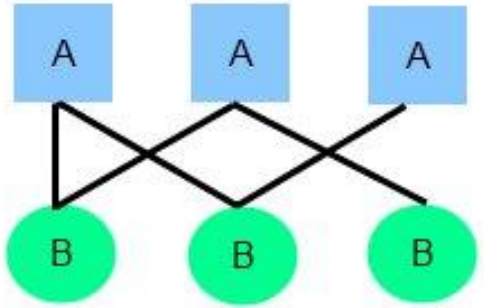
Common relationship

1. one-to-one (1:1)

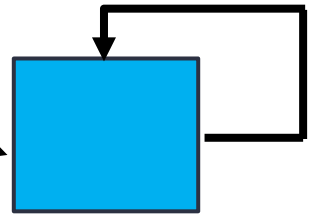
2. one-to-many (1:M)



3. many-to-many (M:N)



4. Self-Referencing (Recursive)



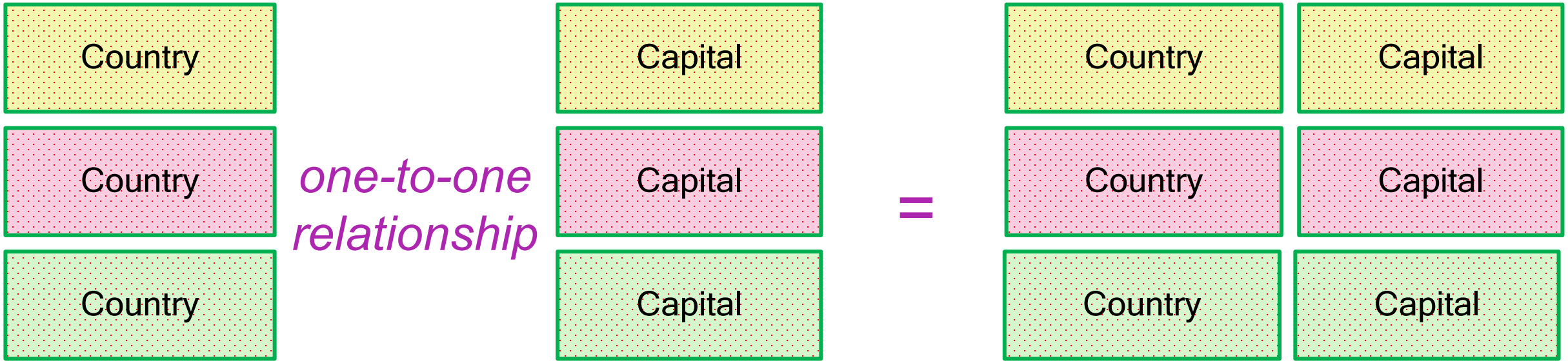
one-to-one relationship

One row in **Table A** corresponds to **one and only one** row in **Table B**.

one-to-one relationship

A *one-to-one* relationship between two tables means that a row in one table can only relate to zero/one row in the table on the other side of their relationship. This is the least common database relationship.

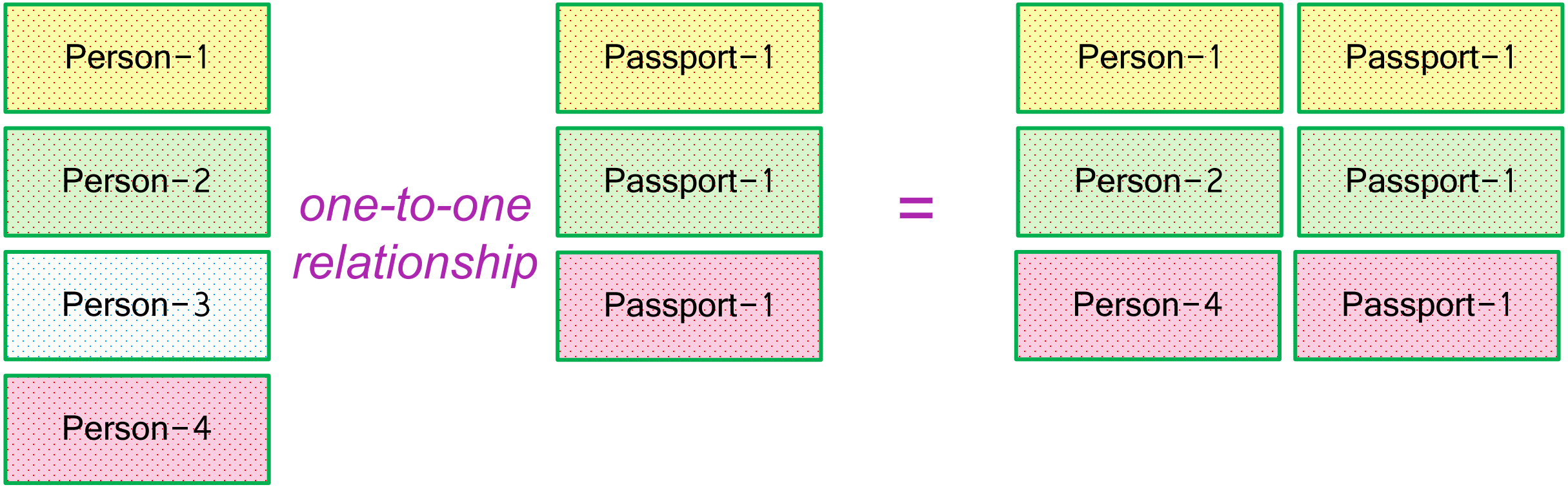
A *one-to-one* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which one element of entity ***R*** may only be linked to zero/one element of entity ***S***, and vice versa.



one-to-one relationship

A *one-to-one* relationship between two tables means that a row in one table can only relate to zero/one row in the table on the other side of their relationship. This is the least common database relationship.

A *one-to-one* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which one element of entity ***R*** may only be linked to zero/one element of entity ***S***, and vice versa.



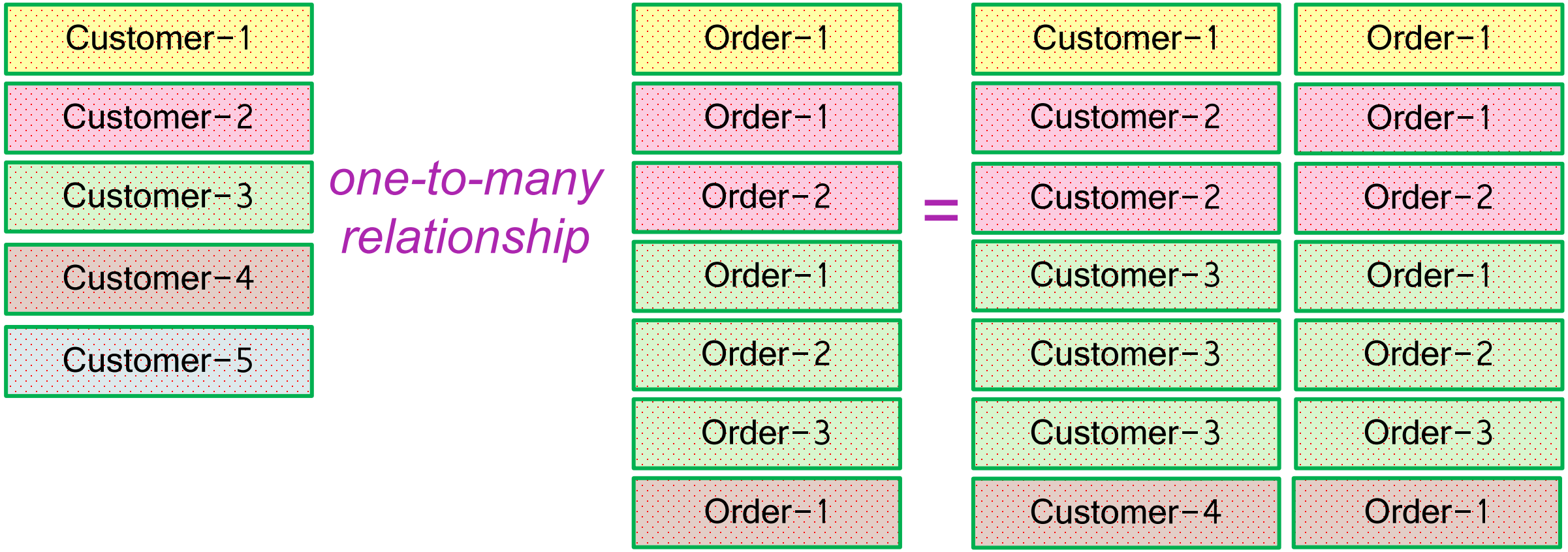
one-to-many relationship

One row in **Table A** can relate to **many rows in Table B**, but **many rows in Table** B **belongs to only one Table A**.

one-to-many relationship

A *one-to-many* relationship between two tables means that a row in one table can have zero or more row in the table on the other side of their relationship.

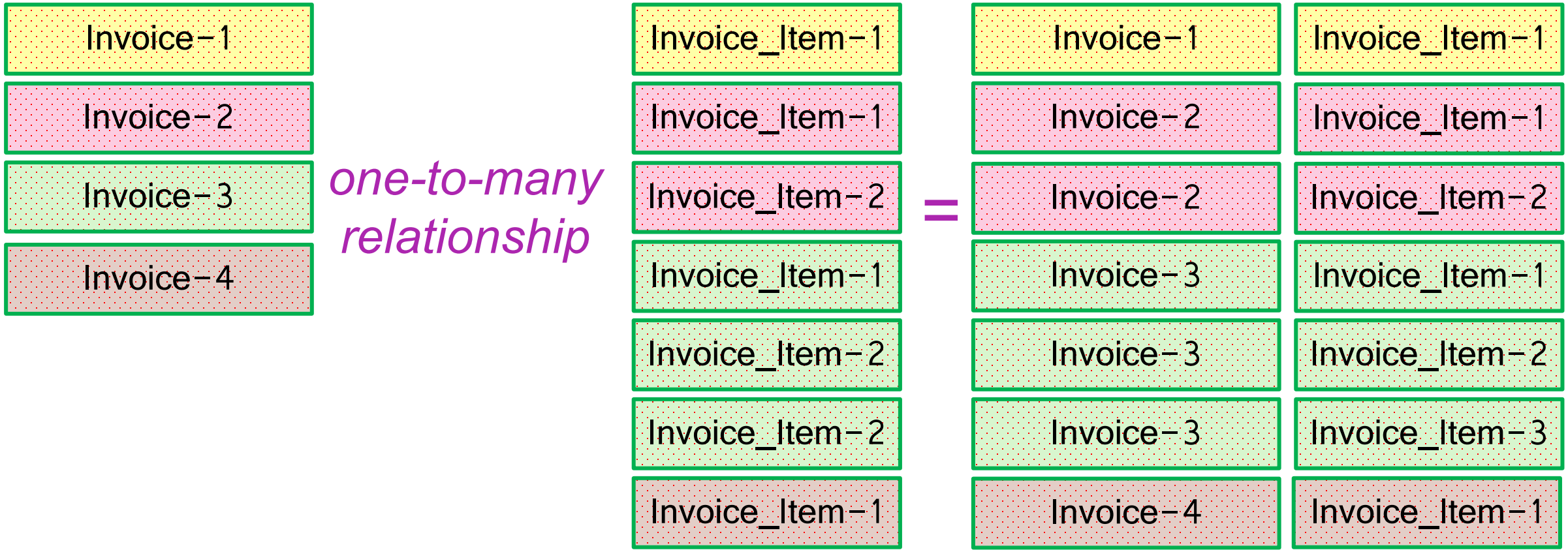
a *one-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which an element of ***R*** may be linked to many elements of ***S***, but a member of ***S*** is linked to only one element of ***R***.



one-to-many relationship

A *one-to-many* relationship between two tables means that a row in one table can have one or more row in the table on the other side of their relationship.

a *one-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which an element of ***R*** may be linked to many elements of ***S***, but a member of ***S*** is linked to only one element of ***R***.

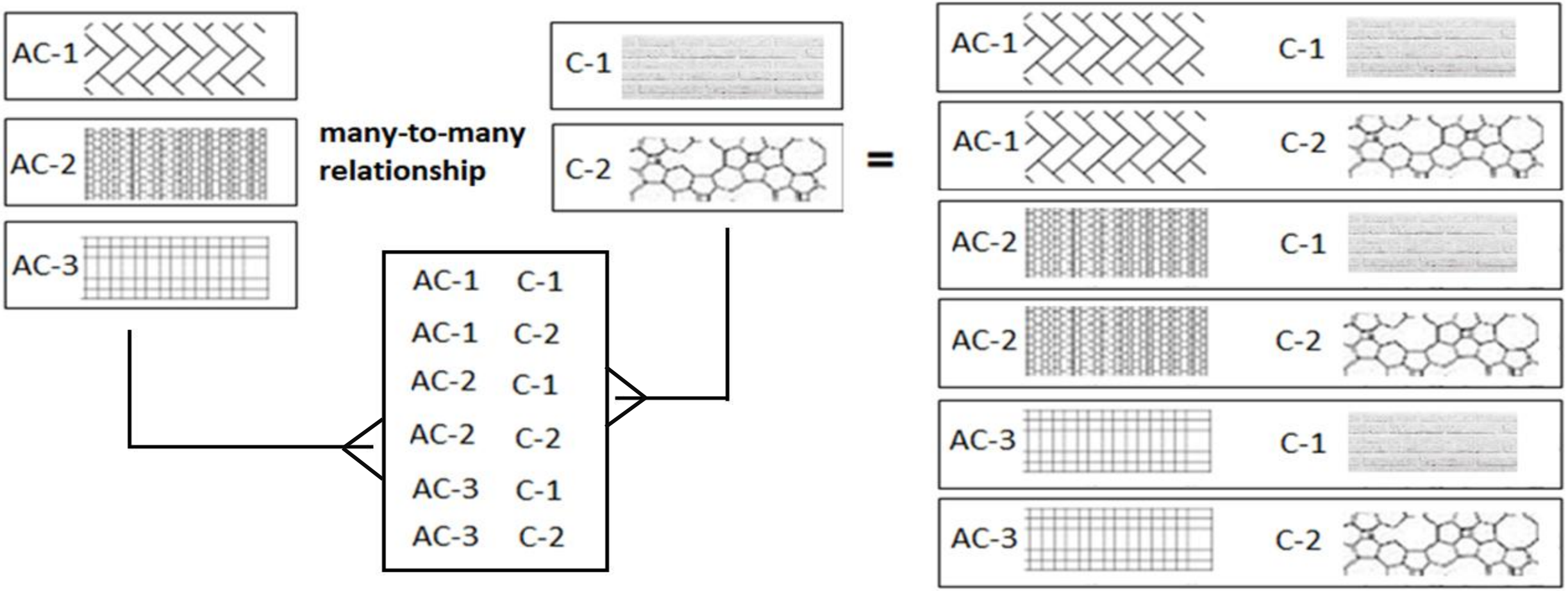


many-to-many relationship

One row in **Table A** can relate to **many rows in Table B**, and one row in **Table B** can also relate to **many rows in Table A**

many-to-many relationship

A *many-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which ***R*** may contain a parent instance for which there are many children in ***S*** and vice versa.

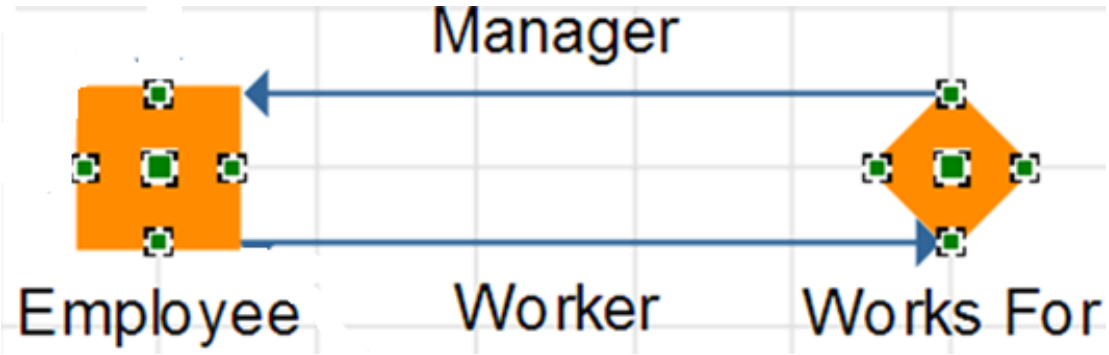


self-referencing relationship

An entity (table) is **related to itself** using a **foreign key** that points to its own primary key.

self-referencing relationship

A *"self-referencing"* or *"recursive"* relationship in databases or data structures means that a record within a table can reference another record in the same table.



Product Categories and Subcategories

CategoryID	CategoryName	ParentCategoryID
1	Electronics	NULL
2	Phones	1
3	Laptops	1
4	Smartphones	2
5	Gaming Laptops	3

MySQL is the most popular **Open Source** Relational Database Management System.

MySQL was created by a Swedish company - MySQL AB that was founded in 1995. It was acquired by Sun Microsystems in 2008; Sun was in turn acquired by Oracle Corporation in 2010.

When you use MySQL, you're actually using at least two programmes. One program is the MySQL server (*mysqld.exe*) and other program is MySQL client program (*mysql.exe*) that connects to the database server.

What is SQL?

Remember:

what is sql?

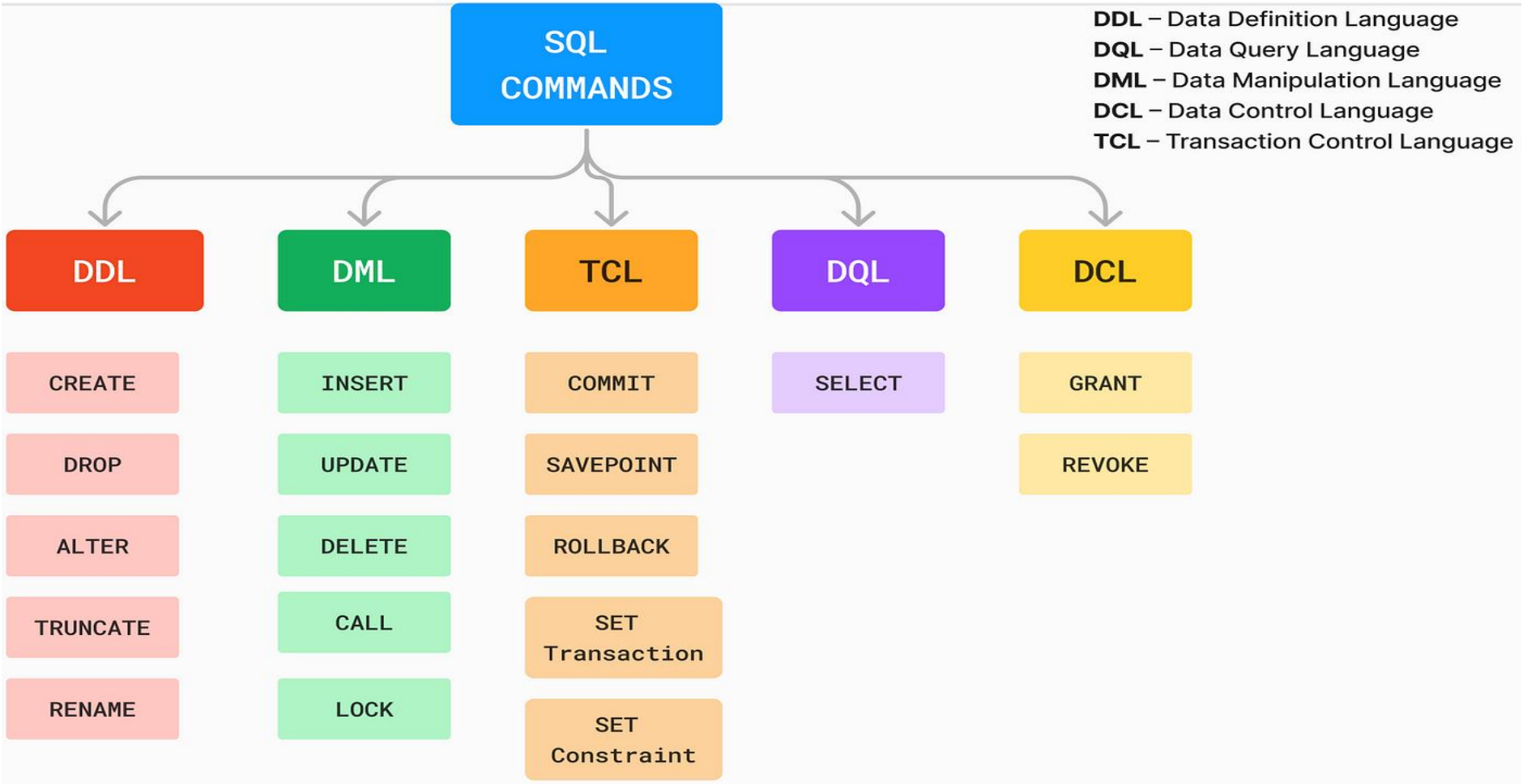
- **EXPLICIT or IMPLICIT commit will commit the data.**

SQL (Structured Query Language) is a database language designed and developed for managing data in relational database management systems (RDBMS). SQL is common language for all Relational Databases.

Remember:

what is sql?

- An ***implicit commit*** occurs ***automatically*** in MySQL ***without the need of COMMIT command***. This means changes made by the SQL statement are immediately saved to the database and ***cannot be rolled back***.
- An ***explicit commit*** is done by the user issuing a ***COMMIT*** command to ***manually save all changes*** made in the current transaction.



comments in mysql

- From a # character to the end of the line.
- From a -- sequence to the end of the line.
- From a /* sequence to the following */ sequence.

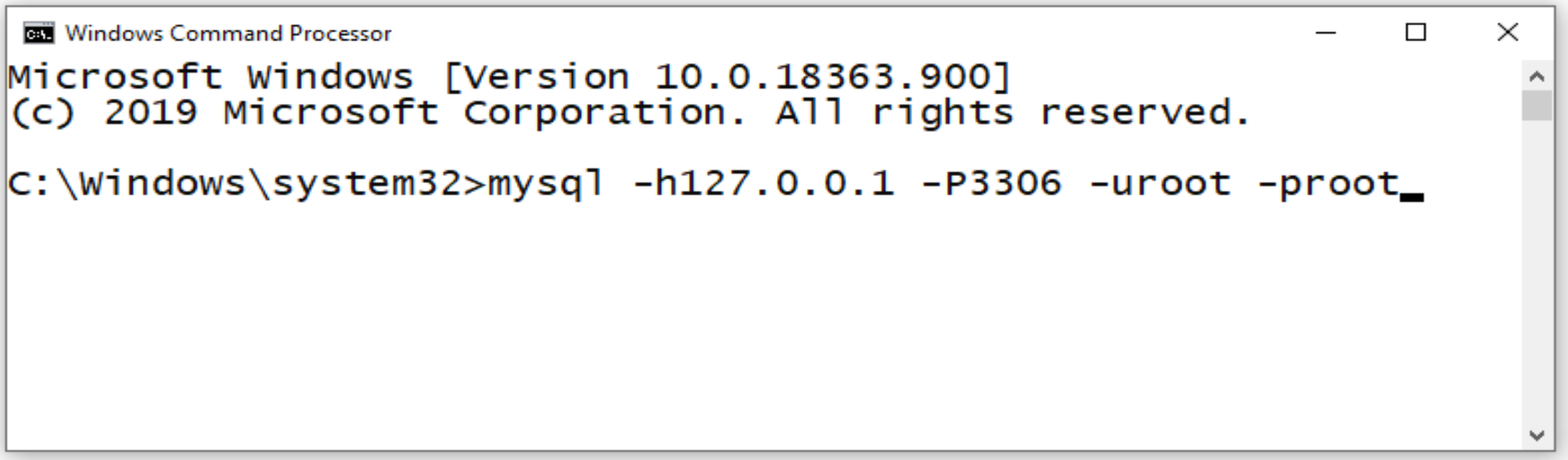
Reconnect to the server	\r
Execute a system shell command	\!
Exit mysql	\q
Change your mysql prompt.	prompt str or \R str

Login to MySQL

Default port for MySQL Server: 3306

login

- C:\> mysql -hlocalhost -P3307 -uroot -p
- C:\> mysql -h127.0.0.1 -P3307 -uroot -p [*database_name*]
- C:\> mysql -h192.168.100.14 -P3307 -uroot -psaleel [*database_name*]
- C:\> mysql --host localhost --port 3306 --user root --password=ROOT [*database_name*]
- C:\> mysql --host=localhost --port=3306 --user=root --password=ROOT [*database_name*]



- Before MySQL version 5.5, MyISAM is the default storage engine.
- From version 5.5, MySQL uses InnoDB as the default storage engine.

STORAGE ENGINES

A storage engine is a software module that a database management system uses to create, read, update data from a database. There are two types of storage engines in MySQL: **transactional** and **non-transactional.**

Storage Engine	File on disk
MEMORY	Data is not stores on the disk
InnoDB	.idb (data and index)
MyISAM	MYD (data), .MYI (index)
CSV	.CSV (data), CSM (metadata)

SHOW ENGINES Syntax

For MySQL 5.5 and later, the default storage engine is *InnoDB*. The default storage engine for MySQL prior to version 5.5 was *MyISAM*.

SHOW [STORAGE] ENGINES

SHOW ENGINES displays status information about the server's storage engines. This is particularly useful for checking whether a storage engine is supported, or to see what the default engine is.

INFORMATION_SCHEMA.ENGINES

- show engines;
- show STORAGE engines;

INFORMATION_SCHEMA provides access to database metadata, information about the MySQL server such as the name of a database or table, the data type of a column, or access privileges.

SHOW DATABASES

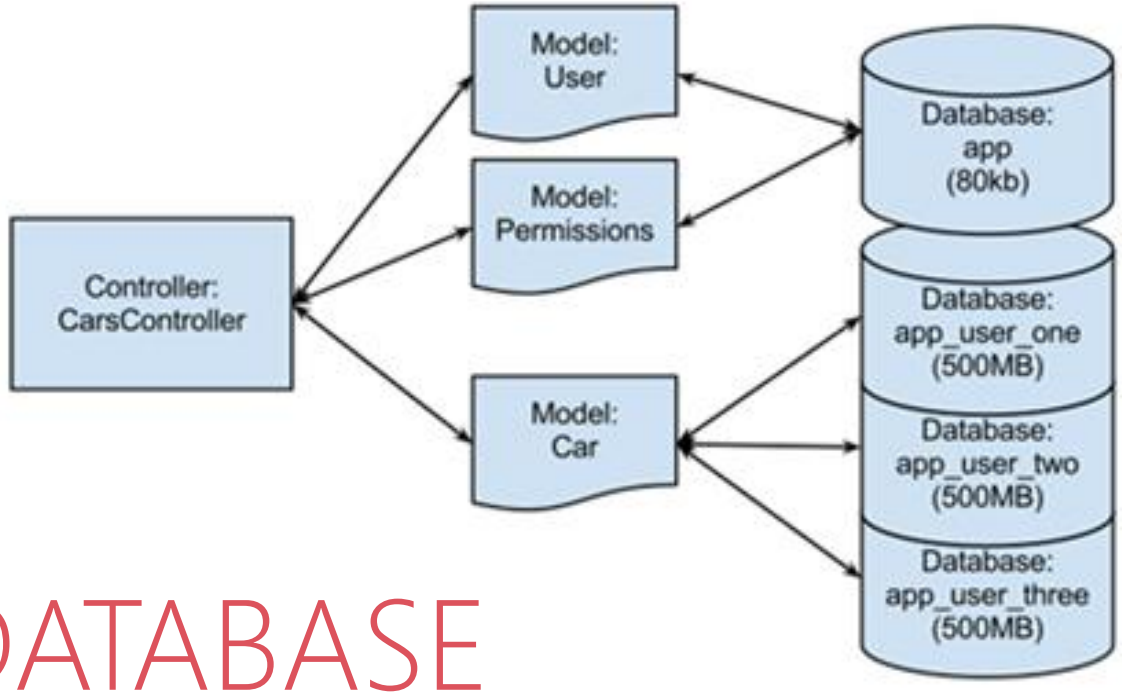
SHOW DATABASES Syntax

SHOW { DATABASES | SCHEMAS } [LIKE '*pattern*' | WHERE *expr*]

SHOW SCHEMAS is a synonym for SHOW DATABASES.

```
SHOW DATABASES;
SHOW SCHEMAS;
SHOW DATABASES LIKE 'U%';
SHOW SCHEMAS LIKE 'U%';
```

NULL means “no database is selected”. Issue the ***USE dbName*** command to select the database.



USE DATABASE

The **USE** ***db_name*** statement tells MySQL to use the db_name database as the default (current) database for subsequent statements. The database remains the default until the end of the session or another **USE** statement is issued.

USE DATABASE Syntax

USE *db_name*
\U *db_name*

Note:

- **USE, does not require a semicolon.**
- **USE must be followed by a database name.**

USE db1

\U db1

CREATE DATABASE
ALTER DATABASE

create / alter database

CREATE DATABASE creates a database with the given name. To use this statement, you need the CREATE privilege for the database.

CREATE { DATABASE | SCHEMA } [IF NOT EXISTS] *db_name*

ALTER { DATABASE | SCHEMA } [*db_name*] READ ONLY [=] { 0 | 1}

CREATE SCHEMA is a synonym for CREATE DATABASE.

- CREATE DATABASE db1;
- CREATE DATABASE IF NOT EXISTS db1;

- ALTER DATABASE db1 READ ONLY = 0; // is in read write mode.
- ALTER DATABASE db1 READ ONLY = 1; // is in read only mode.

Note:

- It is ***not*** possible to Create, Alter, Drop any object, and Write (Insert, Update, and Delete rows) in a read-only database.
- TEMPORARY tables; it is possible to create, alter, drop, and write (Insert, Update, and Delete rows) to TEMPORARY tables in a read-only database.

DROP DATABASE

If the default database is dropped, the default database is unset (the DATABASE() function returns NULL).

drop database

DROP DATABASE drops all tables in the database and deletes the database. Be very careful with this statement! To use DROP DATABASE, you need the DROP privilege on the database.

DROP { DATABASE | SCHEMA } [IF EXISTS] *db_name*

DROP SCHEMA is a synonym for DROP DATABASE.

```
DROP DATABASE db1;

DROP DATABASE IF EXISTS db1;
```

Information Functions

Use these function with SELECT statement

information function

- **FOUND_ROWS()** - For a SELECT with a LIMIT clause, the number of rows that would be returned were there no LIMIT clause

e.g.

  - SELECT * FORM emp;
  - SELECT **FOUND_ROWS()**;

- **LAST_INSERT_ID()** - Value of the AUTOINCREMENT column for the last INSERT
- **ROW_COUNT()** - The number of rows updated.

e.g.

  - SELECT * FORM emp;
  - SELECT **ROW_COUNT();**

- **DATABASE()** - Return the default (current) database name

e.g.

  - SELECT **DATABASE()**;

- **SCHEMA()** - Synonym for DATABASE()

information function

- **CURRENT_USER(), CURRENT_USER** - The authenticated user name and host name
- **USER()** - The user name and host name provided by the client

e.g.

  - SELECT **USER()**;

- **SESSION_USER()** - Synonym for USER()
- **SYSTEM_USER()** - Synonym for USER()
- **VERSION()** - Return a string that indicates the MySQL server version

e.g.

  - SELECT **VERSION()**;

- **CONNECTION_ID()** - Return the connection ID (thread ID) for the connection

The **char** is a fixed-length character data type,
The **varchar** is a variable-length character data type.

```
CREATE TABLE temp (c1 CHAR(10), c2 VARCHAR(10));
INSERT INTO temp VALUES('SALEEL', 'SALEEL');
SELECT * FROM temp WHERE c1 LIKE 'SALEEL';
```

datatypes

ENAME CHAR (10)	S	A	L	E	E	L					LENGTH -> 10
ENAME VARCHAR2(10)	S	A	L	E	E	L					LENGTH -> 6

In MySQL

When CHAR values are retrieved, the trailing spaces are removed (unless the ***PAD_CHAR_TO_FULL_LENGTH*** SQL mode is enabled)

ENAME CHAR (10)	S	A	L	E	E	L					LENGTH -> 6
ENAME VARCHAR(10)	S	A	L	E	E	L					LENGTH -> 6

Note:

The BINARY and VARBINARY types are similar to CHAR and VARCHAR, except that they store binary strings rather than nonbinary strings. That is, they store byte strings rather than character strings.

quantity VARCHAR(20) -- values: 20, '20', 'twenty'

datatype - string

Datatypes	Size	Description
CHAR [(length)]	0-255	
VARCHAR (length)	0 to 65,535	The maximum row size (65,535 bytes, which is shared among all columns.
TINYTEXT [(length)]	$(2^8 - 1)$ bytes	
TEXT [(length)]	$(2^{16} - 1)$ bytes	65,535 bytes ~ 64kb
MEDIUMTEXT [(length)]	$(2^{24} - 1)$ bytes	16,777,215 bytes ~16MB
LONGTEXT [(length)]	$(2^{32} - 1)$ bytes	4,294,967,295 bytes ~4GB
ENUM('value1', 'value2',...)	65,535 members	
SET('value1', 'value2',...)	64 members	
BINARY[(length)]	255	
VARBINARY(length)		

By default, trailing spaces are trimmed from CHAR column values on retrieval. If ***PAD_CHAR_TO_FULL_LENGTH*** is enabled, trimming does not occur and retrieved CHAR values are padded to their full length.

- *SET sql_mode = '';*
- *SET sql_mode = 'PAD_CHAR_TO_FULL_LENGTH';*

example of char and varchar

Datatypes	Size	Description
CHAR [(length)]	0-255	
VARCHAR (length)	0 to 65,535	The maximum row size (65,535 bytes, which is shared among all columns.

Try Out

- CREATE TABLE x (x1 CHAR(4), x2 VARCHAR(4));
- INSERT INTO x VALUE('', '');
- INSERT INTO x VALUE('ab', 'ab');
- INSERT INTO x VALUE('abcd', 'abcd');
- SELECT x1, LENGTH(x1), x2, LENGTH(x2) FROM x;
- SET sql_mode = 'PAD_CHAR_TO_FULL_LENGTH';
- SELECT x1, LENGTH(x1), x2, LENGTH(x2) FROM x;
- SET sql_mode = '';
- SELECT x1, LENGTH(x1), x2, LENGTH(x2) FROM x;

* In CHAR, if a table contains value 'a', an attempt to store 'a ' causes a duplicate-key error.

- CREATE TABLE x (x1 CHAR(4) PRIMARY KEY, x2 VARCHAR(4));
- INSERT INTO x VALUE('a', 'a');
- INSERT INTO x VALUE('a ', 'a ');

- CREATE TABLE x (x1 CHAR(4), x2 VARCHAR(4) PRIMARY KEY);
- INSERT INTO x VALUE('a', 'a');
- INSERT INTO x VALUE('a ', 'a ');

datatype - numeric

- CREATE TABLE z1 (c1 ZEROFILL); // then Insert few records
- CREATE TABLE z1 (c1 INT(4) ZEROFILL);

Datatypes	Size	Description
TINYINT	1 byte	-128 to +127 **(The unsigned range is 0 to 255).**
SMALLINT [(length)]	2 bytes	-32768 to 32767. **(The unsigned range is 0 to 65535).**
MEDIUMINT [(length)]	3 bytes	-8388608 to 8388607. **(The unsigned range is 0 to 16777215).**
INT, INTEGER [(length)]	4 bytes	-2147483648 to 2147483647. **(The unsigned range is 0 to 4294967295).**
BIGINT [(length)]	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
FLOAT [(length[,decimals])]	4 bytes	**FLOAT(255,30)**
DOUBLE [PRECISION] [(length[,decimals])], REAL [(length[,decimals])]	8 bytes	**REAL(255,30) / DOUBLE(255,30)** REAL will get converted to DOUBLE
DECIMAL [(length[,decimals])], NUMERIC [(length[,decimals])]		**DECIMAL(65,30) / NUMERIC(65,30)** NUMERIC will get converted in DECIMAL

For: float(M,D), double(M,D) or decimal(M,D), M must be >= D

Here, **(M,D)** means than values can be stored with up to ***M*** digits in total, of which ***D*** digits may be after the decimal point.

UNSIGNED prohibits negative values.

datatype – date and time

Datatypes	Size	Description
YEAR	1 byte	YYYY
DATE	3 bytes	YYYY-MM-DD
TIME	3 bytes	HH:MM:SS
DATETIME	8 bytes	YYYY-MM-DD hh:mm:ss

datatype – boolean

A value of zero is considered false. Nonzero values are considered true.

ColumnName BOOLEAN

```
CREATE TABLE tasks ( id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255) NOT NULL, completed BOOLEAN);
```

- INSERT INTO tasks VALUE(default, 'Task1', 0);
- INSERT INTO tasks VALUE(default, 'Task2', 1);
- INSERT INTO tasks VALUE(default, 'Task3', False);
- INSERT INTO tasks VALUE(default, 'Task4', True);
- INSERT INTO tasks VALUE(default, 'Task5', NULL);
- INSERT INTO tasks VALUE(default, 'Task6', default);
- INSERT INTO tasks VALUE(default, 'Task7’, 1 > 2);
- INSERT INTO tasks VALUE(default, 'Task8’, 1 < 2);
- INSERT INTO tasks VALUE(default, 'Task9', 12);
- INSERT INTO tasks VALUE(default, 'Task10', 58);
- INSERT INTO tasks VALUE(default, 'Task11', .75);
- INSERT INTO tasks VALUE(default, 'Task12', .15);
- INSERT INTO tasks VALUE(default, 'Task13', 'a' = 'a');

Note:

- BOOL and BOOLEAN are **synonym of TINYINT(1)**

Remember:

MySQL performs type conversion when comparing values of different types.

- 'b' is a string. When compared to a number (0), MySQL converts the string to a number.
- 'b' → numeric conversion = 0 (since it’s not a valid number).

Use a CREATE TABLE statement to specify the layout of your table.

create table

Use a **CREATE TABLE** statement to specify the layout of your table.

Note:

- **USER TABLES**: This is a collection of tables created and maintained by the user. Contain USER information.
- **DATA DICTIONARY**: This is a collection of tables created and maintained by the MySQL Server. It contains database information. All data dictionary tables are owned by the SYS user.

create table

Use a **CREATE TABLE** statement to specify the layout of your table.

- CREATE TABLE `123` (c1 INT, c2 VARCHAR(10));
- CREATE TABLE `Order Details` (c1 INT, c2 VARCHAR(10));

Remember:

- by default, tables are created in the default database, using the InnoDB storage engine.
- table name should not begin with a number or special symbols.
- table name can start with _table_name (underscore) or $table_name (dollar sign)
- table name and column name can have max 64 char.
- multiple words as table_name is invalid, if you want to give multiple words as table_name then give it in `table_name` (backtick)
- error occurs if the table exists.
- error occurs if there is no default database.
- error occurs if the database does not exist.

create table

syntax

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
   (create_defineation, . . .)
   [table_options]
   [partition_options]

create_definition:
   col_name column_definition

column_definition:
   data_type [NOT NULL | NULL] [DEFAULT default_value]
     [AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]
     [reference_definition]
  | data_type [GENERATED ALWAYS] AS (expression)  [VIRTUAL]
    [VISIBLE | INVISIBLE]

table_options:
AUTO_INCREMENT = <number>  // must be used with AUTO_INCREMENT definition
ENGINE [=] engine_name
```

e.g.

-
```
CREATE TABLE student(
  ID INT,
  firstName VARCHAR(45),
  lastName VARCHAR(45),
  DoB DATE,
  emailID VARCHAR(128)
);
```

```
show engines;
set default_storage_engine = memory;
```

- Literals, built-in functions (both deterministic and nondeterministic), and operators are permitted.
- Subqueries, parameters, variables, and stored functions are not permitted.
- An expression default value cannot depend on a column that has the AUTO_INCREMENT attribute.

default value

The DEFAULT specifies a default value for the column.

- `CREATE TABLE temp (c1 INT PRIMARY KEY AUTO_INCREMENT, c2 INT DEFAULT(c1 + c2));  // Error`
- `CREATE TABLE temp (c1 INT, c2 INT DEFAULT(c1 < c2)); // Error`
- `CREATE TABLE temp (c1 INT, c2 INT , c3 INT DEFAULT(c1 < c2)); // OK`

default value

col_name data_type DEFAULT value

The **DEFAULT** specifies a **default** value for the column.

```
CREATE TABLE posts(
  postID INT,
  postTitle VARCHAR(255),
  postDate DATETIME DEFAULT NOW(),
  deleted INT
);
```

Field	Type	Null	Key	Default	Extra
postID	int	YES		NULL	
postTitle	varchar(255)	YES		NULL	
postDate	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
deleted	int	YES		NULL	

version 8.0 and above.

```
CREATE TABLE empl(
  ID INT PRIMARY KEY,
  firstName VARCHAR(45),
  phone INT,
  city VARCHAR(10) DEFAULT 'PUNE',
  salary INT,
  comm INT,
  total INT DEFAULT(salary + comm)
);
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
firstName	varchar(45)	YES		NULL	
phone	int	YES		NULL	
city	varchar(10)	YES		PUNE	
salary	int	YES		NULL	
comm	int	YES		NULL	
total	int	YES		(`salary` + `comm`)	DEFAULT_GENERATED

default value - insert

The **DEFAULT** example.

- CREATE TABLE t(
  c1 INT,
  c2 INT DEFAULT 1,
  c3 INT DEFAULT 3,
  );

	Field	Type	Null	Key	Default	Extra
▶	c1	int	YES		NULL	
	c2	int	YES		1	
	c3	int	YES		3	

  - INSERT INTO t VALUES();
  - INSERT INTO t VALUES(-1, DEFAULT, DEFAULT);
  - INSERT INTO t VALUES(-2, DEFAULT(c2), DEFAULT(c3));
  - INSERT INTO t VALUES(-3, DEFAULT(c3), DEFAULT(c2));

default value - update

The **DEFAULT** example.

- ```
  CREATE TABLE temp(
    c1 INT,
    c2 INT,
    c3 INT DEFAULT(c1 + c2),
    c4 INT DEFAULT(c1 * c2)
  );
  ```

- ```
  INSERT INTO temp (c1, c2, c3, c4) VALUES(1, 1, 1, 1);
  ```
- ```
  INSERT INTO temp (c1, c2, c3, c4) VALUES(2, 2, 2, 2);
  ```
- ```
  UPDATE temp SET c3 = DEFAULT;
  ```
- ```
  UPDATE temp SET c4 = DEFAULT;
  ```
  ```

insert rows

INSERT is used to add a single or multiple tuple to a relation. We must specify the relation name and a list of values for the tuple. **The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command.**

You can insert data using following methods:

- INSERT ... VALUES
- INSERT ... SET
- INSERT ... SELECT

INSERT can violate for any of the four types of constraints.

Important:

- If an attribute value is not of the appropriate data type.
- Entity integrity can be violated if a key value in the new tuple *t* already exists in another tuple in the relation r(R).
- Entity integrity can be violated if any part of the primary key of the new tuple *t* is NULL.
- Referential integrity can be violated if the value of any foreign key in *t* refers to a tuple that does not exist in the referenced relation.

INSERT will also fail in following cases.

Important :

- Your database table has **X** columns, Where as the **VALUES** you are passing are for (**X-1**) or (**X+1**). This mismatch of column-values will giving you the error.
- Inserting a string into a string column that exceeds the column maximum length. Data too long for column error will be raise.
- Inserting data into a column than does not exists, then Unknown column error will raise.
- INSERT INTO *tbl_name* (col1,col2) VALUES(15,col1*2); // is legal.
- INSERT INTO *tbl_name* (col1,col2) VALUES(col2*2,15); // is not legal, because the value for col1 refers to col2, which is assigned after col1.

insert rows using values

dml- insert … values

INSERT inserts new row(s) into an existing table. The INSERT ... VALUES

INSERT [IGNORE] [INTO] tbl_name [PARTITION (*partition_name* [, *partition_name*] ...)] [(*field_name*, . . .)]
{ VALUES | VALUE } [ROW] ({ *expr* | *DEFAULT* }, . . .), [ROW] (. . .), [ROW] . . . [ON DUPLICATE KEY UPDATE assignment_list]

The affected-rows value for an INSERT can be obtained using the ROW_COUNT() function.

```
INSERT INTO DEPT VALUES (1, 'HRD', 'Pune')
```

Column Values

```
INSERT INTO DEPT(ID, NAME, LOC) VALUES (1, 'HRD', 'Pune')
```

Column List

```
INSERT INTO DEPT(ID, NAME, LOC) VALUES (1, 'HRD', 'Baroda'),
 (2,'Sales','Surat'), (3,'Purchase','Pune'), (4,'Account','Mumbai')
```

Inserting multiple rows

Inserting Multiple Rows Using Multiple INSERT

Anti-pattern

- Here,
```
INSERT INTO emp(ename, sal) VALUES('A', 10000);
INSERT INTO emp(ename, sal) VALUES('B', 15000);
INSERT INTO emp(ename, sal) VALUES('C', 12000);
```

Why it's bad:

- Generates high network overhead and slow performance.

Better approach: Bulk Insert

- Here,
```
INSERT INTO emp(ename, sal) VALUES
('A', 10000),
('B', 15000),
('C', 12000);
```

INSERT to Fetch Auto-Increment ID Incorrectly

Anti-pattern

- Here, INSERT INTO dept(deptno) VALUES(1001);
  SELECT MAX(deptno) FROM dept; -- WRONG

Why it's bad:

- MAX(deptno) is not safe if multiple users insert simultaneously.

Better approach:

- Here, SELECT LAST_INSERT_ID();

INSERT Without Using Transactions

Anti-pattern

- Here, INSERT INTO invoice VALUES(. . .);
  INSERT INTO invoice_details VALUES(. . .); -- WRONG

Why it's bad:

- If the ***second*** INSERT command fails, the ***first*** INSERT statement will be executed and a record will be created.

Better approach:

- Here, START TRANSACTION READ WRITE;
  INSERT INTO invoice VALUES(. . .);
  INSERT INTO invoice_details VALUES(. . .);
  COMMIT;

insert rows using select

insert ... select

With INSERT ... SELECT, you can quickly insert many rows into a table from one or many tables.

INSERT [INTO] tbl_name [(*col_name*, . . .)] SELECT . . .

- INSERT INTO dept(deptno) SELECT 1 + 1;
- INSERT INTO dept(deptno) SELECT deptno FROM dept;
- INSERT INTO dept SELECT * FROM dept;
- INSERT INTO dept(SELECT MAX(deptno) + 1, 'HRD', 'BARODA', 'r57px33px' FROM dept);

- set @x := 40;
- INSERT INTO dept VALUES (@x := @x + 1 , 'HRD', 'BARODA', 'r57px33px');

- set @x := (SELECT MAX(deptno) FROM dept);
- INSERT INTO dept VALUES (@x := @x + 1 , 'HRD', 'BARODA', 'r57px33px');

SELECT statement...

```
SELECT what_to_select
FROM which_table
WHERE conditions_to_satisfy;
```

SELECT CLAUSE

The **SELECT** statement retrieves or extracts data from tables in the database.

- You can use one or more tables separated by comma to extract data.
- You can fetch one or more fields/columns in a single SELECT command.
- You can specify star (*) in place of fields. In this case, SELECT will return all the fields.
- SELECT can also be used to retrieve rows computed without reference to any table e.g. SELECT 1 + 2;

Capabilities of SELECT Statement

1. SELECTION
2. PROJECTION
3. JOINING

Capabilities of SELECT Statement

➢ SELECTION

Selection capability in SQL is to choose the record's/row's/tuple's in a table that you want to return by a query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ PROJECTION

Projection capability in SQL to choose the column's/attribute's/field's in a table that you want to return by your query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ JOINING

Join capability in SQL to bring together data that is stored in different tables by creating a link between them.

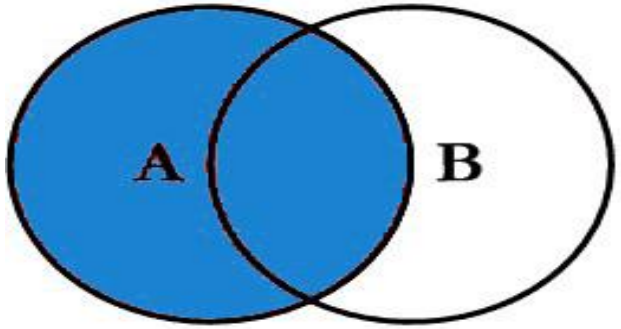
R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	20
2	Janhavi	Sales	1994-12-20	10
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	20
5	Ketan	Sales	1994-01-01	30

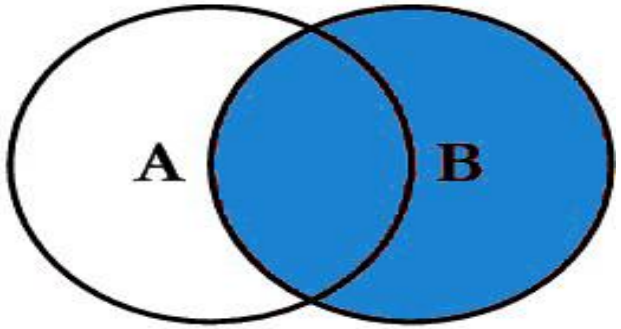
S

DEPTNO	DNAME	LOC
10	HRD	PUNE
20	SALES	BARODA
40	PURCHASE	SURAT

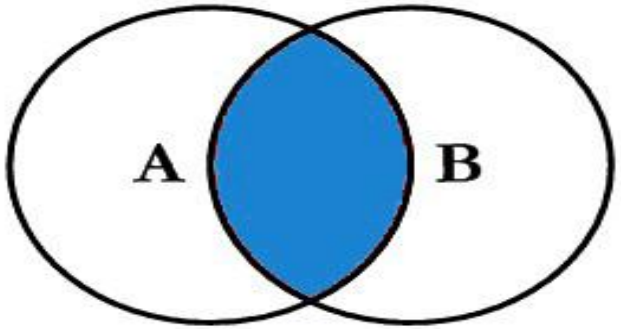
SQL JOINS



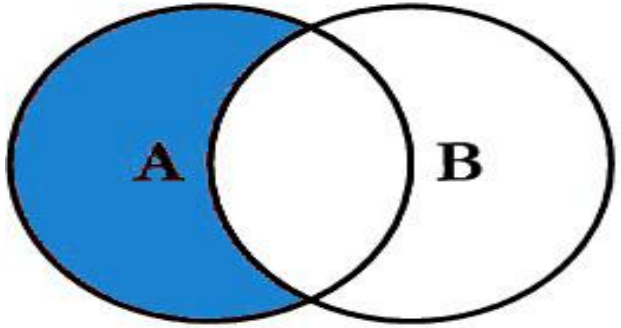

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



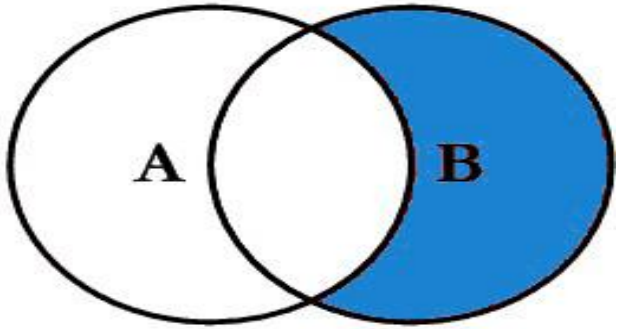

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



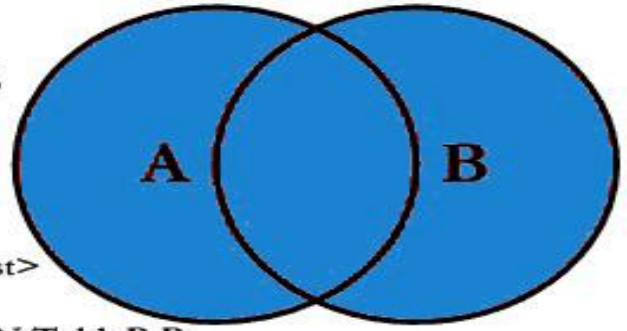

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



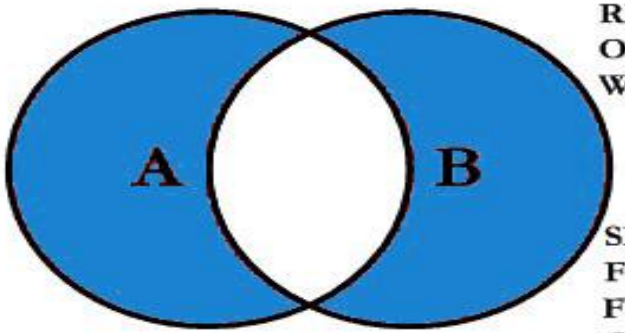

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

SELECTION Process

SELECT * FROM <table_references>

selection-list | field-list | column-list

Remember:

- Here, " * " is known as metacharacter (all columns)

PROJECTION Process

SELECT column-list FROM <table_references>

selection-list | field-list | column-list

Remember:

- Position of columns in SELECT statement will determine the position of columns in the output (as per user requirements)

In a **SET** statement, = is treated identically to :=

```
SELECT ename, job, sal, sal * 1.1, sal * 1.25 FROM emp;
```

Salary increased by **10%**

Salary increased by **25%**

single-table update

UPDATE is used to change/modify the values of some attributes of one or more selected tuples.

single-table update

The UPDATE statement updates columns of existing rows in the named table with new values. The SET clause indicates which columns to modify and the values they should be given. The **WHERE** clause, if given, specifies the conditions that identify which rows to update. With **no WHERE** clause, all rows are updated. If the **ORDER BY** clause is specified, the rows are updated in the order that is specified. The **LIMIT** clause places a limit on the number of rows that can be updated.

```
UPDATE tbl_name SET col_name1 = { expr1 | DEFAULT | query } [, col_name2 = { expr2 | DEFAULT | query } ] . . .
  [WHERE where_condition]  [ORDER BY . . .]  [LIMIT row_count]
```

- UPDATE temp SET dname = 'new_value' LIMIT 2;
- UPDATE temp SET c1 = 'new_value' ORDER BY loc LIMIT 2;
- UPDATE temp SET c1 := 'new_value' WHERE deptno < 50;
- UPDATE temp SET c1 := 'new_value' WHERE deptno < 50 LIMIT 2;
- ALTER TABLE dept ADD SUMSALARY INT;
- UPDATE dept SET sumsalary = (SELECT SUM(sal) FROM emp WHERE emp.deptno = dept.deptno GROUP BY emp.deptno);
- UPDATE candidate SET totalvotes = (SELECT COUNT(*) FROM votes WHERE candidate.id = votes.candidateID GROUP BY votes.candidateID);
- UPDATE duplicate SET id = (SELECT @cnt := @cnt + 1);

single-table delete

DELETE is used to delete tuples from a relation.

single-table delete

The DELETE statement deletes rows from tbl_name and returns the number of deleted rows. To check the number of deleted rows, call the *ROW_COUNT()* function. The optional WHERE clause identify which rows to delete. With no WHERE clause, all rows are deleted. If the ORDER BY clause is specified, the rows are deleted in the order that is specified. The LIMIT clause places a limit on the number of rows that can be deleted.

DELETE FROM tbl_name [WHERE *where_condition*]

Note:

- LIMIT clauses apply to single-table deletes, but not multi-table deletes.
- DELETE FROM temp;
- DELETE FROM temp ORDER BY loc LIMIT 2;
- DELETE FROM temp WHERE deptno < 50;
- DELETE FROM temp WHERE deptno < 50 LIMIT 2;

auto_increment column

The **AUTO_INCREMENT** attribute can be used to generate a unique number/identity for new rows.

Gap occurs when :- INSERT fails or is rolled back

auto_increment

col_name data_type AUTO_INCREMENT [UNIQUE [KEY] | [PRIMARY] KEY]

IDENTITY is a synonym to the ***LAST_INSERT_ID*** variable.

Remember:

- There can be **only one** AUTO_INCREMENT column per table.
- The column defined with AUTO_INCREMENT **must be indexed**.
- it cannot have a DEFAULT value.
- it contains only **positive values**.
- AUTO_INCREMENT works only with **integer types** (TINYINT, SMALLINT, INT, BIGINT . . .)
- when you insert a value of NULL or 0 into AUTO_INCREMENT column, it generates **next value**.
- If a row is deleted or a transaction is rolled back, the auto_increment number is not reused.
- Truncating a table **resets** AUTO_INCREMENT back to 1.
- If you insert multiple rows in one INSERT, each row gets a **different value**.
- *use LAST_INSERT_ID*() function to find the row that contains the most recent AUTO_INCREMENT value.

- CREATE TABLE animals(id INT NOT NULL AUTO_INCREMENT, breed INT, PRIMARY KEY (id, breed)); //valid
- CREATE TABLE animals(id NT NOT NULL, breed INT AUTO_INCREMENT, PRIMARY KEY (id, breed)); //invalid

- SELECT @@IDENTITY
- SELECT LAST_INSERT_ID()
- SET INSERT_ID = 7

- CREATE TABLE posts (
  c1 INT UNIQUE KEY AUTO_INCREMENT,
  c2 VARCHAR(20)
) AUTO_INCREMENT = 2; // auto_number will start with value 2.

auto_increment

The **auto_increment** specifies a **auto_increment** value for the column.

- CREATE TABLE posts(
  postID INT AUTO_INCREMENT UNIQUE KEY,
  postTitle VARCHAR(255),
  postDate DATETIME DEFAULT NOW(),
  deleted INT
  );

Field	Type	Null	Key	Default	Extra
postID	int	NO	PRI	NULL	auto_increment
postTitle	varchar(255)	YES		NULL	
postDate	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
deleted	int	YES		NULL	

- CREATE TABLE comments(
  commentID INT AUTO_INCREMENT PRIMARY KEY,
  comment TEXT,
  commentDate DATETIME DEFAULT NOW(),
  deleted INT
  );

Field	Type	Null	Key	Default	Extra
commentID	int	NO	PRI	NULL	auto_increment
comment	text	YES		NULL	
commentDate	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
deleted	int	YES		NULL	

- CREATE TABLE animals(id INT NOT NULL AUTO_INCREMENT, breed INT, PRIMARY KEY (id, breed)); //valid
- CREATE TABLE animals(id NT NOT NULL, breed INT AUTO_INCREMENT, PRIMARY KEY (id, breed)); //invalid

- **CREATE TABLE . . . LIKE . . .**, the destination table *preserves generated column information* from the original table.
- **CREATE TABLE . . . SELECT . . .**, the destination table *does not preserves generated column information* from the original table.

generated column

A SQL generated column is a type of column that stores values calculated from an expression applied to data in other columns of the same table. The value of a generated column cannot be altered manually and is automatically updated whenever the data it depends on changes.

Remember:

- Stored functions and user-defined functions are not permitted.
- Stored procedure and function with parameters are not permitted.
- Variables (system variables, user-defined variables, and stored program local variables) are not permitted.
- Subqueries are not permitted.
- You cannot use a generated column in a DEFAULT clause.
- The AUTO_INCREMENT attribute cannot be used in a generated column definition.
- You **cannot insert/update** values into a generated column directly.
- Triggers cannot use NEW.COL_NAME or use OLD.COL_NAME to refer to generated columns.
- Stored column cannot be converted to virtual column and virtual column cannot be converted to stored column.
- Generated column can be made as invisible column.

Note:

- The expression can contain literals, built-in functions with no parameters, operators, or references to any column within the same table. If you use a function, it must be scalar and deterministic.

virtual column - generated always

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END) [VIRTUAL | STORED]

- **VIRTUAL**: Column values are not stored, but are evaluated when rows are read, immediately after any BEFORE triggers. A virtual column takes no storage.
- **STORED**: Column values are evaluated and stored when rows are inserted or updated. A stored column does require storage space and can be indexed.

Note:

- The default is **VIRTUAL** if neither keyword is specified.

Field	Type	Null	Key	Default	Extra
productCode	int	NO	PRI	NULL	auto_increment
productName	varchar(45)	YES		NULL	
productVendor	varchar(45)	YES		NULL	
productDescription	text	YES		NULL	
quantityInStock	int	YES		NULL	
buyPrice	float	YES		NULL	
stockValue	float	YES		NULL	VIRTUAL GENERATED

- CREATE TABLE product(
  productCode INT AUTO_INCREMENT PRIMARY KEY,
  productName VARCHAR(45),
  productVendor VARCHAR(45),
  productDescription TEXT,
  quantityInStock INT,
  buyPrice FLOAT,
  stockValue FLOAT GENERATED ALWAYS AS(quantityInStock * buyPrice) VIRTUAL
  );

generated always as column with case when . . . end

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END)

- CREATE TABLE loan(

```
loanID BIGINT PRIMARY KEY AUTO_INCREMENT,
bankName VARCHAR(50),
accountNumber VARCHAR(30),
amount INT,
riskStatus VARCHAR(30) GENERATED ALWAYS AS (
CASE
   WHEN amount < 1000 THEN 'Low Risk'
   WHEN amount BETWEEN 1000 AND 2000 THEN 'Above Average Risk'
   ELSE 'High Risk’
END));
```

generated always as column with ENUM and case when . . . end

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END)

- CREATE TABLE patientAdmission(

```
  admissionID INT PRIMARY KEY,
  patientName VARCHAR(10),
  referred_by ENUM('ref1', 'ref2', 'ref3', 'ref4', 'ref5', 'ref9'),
  admissionType VARCHAR(40) GENERATED ALWAYS AS (
  CASE
     WHEN referred_by = 'ref1' THEN 'Emergency'
     WHEN referred_by = 'ref2' THEN 'Urgent'
     WHEN referred_by = 'ref3' THEN 'Elective'
     WHEN referred_by = 'ref4' THEN 'Newborn'
     WHEN referred_by = 'ref5' THEN 'Trauma Center'
     WHEN referred_by = 'ref9' THEN 'Information Not Available'
  END));
```

generated always as column with ENUM and case when . . . end

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END)

- ```
  CREATE TABLE ticketReservation (
    ticketID INT PRIMARY KEY AUTO_INCREMENT,
    reservationType ENUM('GN', 'TQ','LD','FT', 'HP', 'OQ'),
    description VARCHAR(100) GENERATED ALWAYS AS (
    CASE
      WHEN reservationType = 'GN' THEN 'General Quota'
      WHEN reservationType = 'TQ' THEN 'Tatkal Quota'
      WHEN reservationType = 'LD' THEN 'Ladies Quota'
      WHEN reservationType = 'FT' THEN 'Foreign Tourists Quota'
      WHEN reservationType = 'HP' THEN 'Physically Handicapped'
      WHEN reservationType = 'OQ' THEN 'Headquarters/High Official, Parliament House, Defence, Duty Pass Quota'
   END));
  ```
  ```

generated always as column with case when . . . end

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END)

- CREATE TABLE hostelBilling(

```
billingID INT AUTO_INCREMENT PRIMARY KEY,
studentID INT,
roomRent INT,
messCharges INT,
laundryCharges INT,
discountPercent INT,
amount INT GENERATED ALWAYS AS ((roomRent + messCharges + laundryCharges) * discountPercent / 100,
amenities varchar(150) GENERATED ALWAYS AS (
CASE
  WHEN (roomRent + messCharges + laundryCharges) BETWEEN 5000 AND 6000 THEN 'Wi-Fi & Bed-sheets'
   WHEN (roomRent + messCharges + laundryCharges) BETWEEN 6001 AND 8000 THEN 'Wi-Fi, Bed-sheets, Reading
        Room & Lockers'
  WHEN (roomRent + messCharges + laundryCharges) BETWEEN 8001 AND 10000 THEN 'Wi-Fi, Bed-sheets, Reading
        Room, Lockers and Ac, Fridge, Induction stove'
END));
```

visible / invisible columns

Columns are visible by default. To explicitly specify visibility for a new column, use a VISIBLE or INVISIBLE keyword as part of the column definition for CREATE TABLE or ALTER TABLE.

Note:

- An invisible column is normally hidden to queries, but can be accessed if explicitly referenced. Prior to MySQL 8.0.23, all columns are visible.
- A table must have at least one visible column. Attempting to make all columns invisible produces an error.
- SELECT * queries by default does not include invisible columns.
- Invisible columns can be indexed and used in constraints (PK, UNIQUE, FK).
- You can change visibility with ALTER TABLE.
- Both STORED and VIRTUAL generated columns can also be declared VISIBLE or INVISIBLE.

invisible column

col_name data_type INVISIBLE

- CREATE TABLE employee(
  ID INT AUTO_INCREMENT PRIMARY KEY,
  firstName VARCHAR(40),
  salary INT,
  commission INT,
  total INT DEFAULT(salary + commission) INVISIBLE
  tax INT GENERATED ALWAYS AS (total * .25) VIRTUAL INVISIBLE
) ;

- CREATE TABLE employee(
  ID INT PRIMARY KEY AUTO_INCREMENT INVISIBLE ,
  firstName VARCHAR(40)
) ;

- INSERT INTO employee(firstName, salary, commission) VALUES('ram', 4700, -700);
- INSERT INTO employee(firstName, salary, commission) VALUES('pankaj', 3400, NULL);
- INSERT INTO employee(firstName, salary, commission) VALUES('rajan', 3200, 250);
- INSERT INTO employee(firstName, salary, commission) VALUES('ninad', 2600, 0);
- INSERT INTO employee(firstName, salary, commission) VALUES('omkar', 4500, 300);
- SELECT * FROM employee;

- ALTER TABLE employee MODIFY total INT VISIBLE;
- ALTER TABLE employee MODIFY total INT INVISIBLE;

varbinary column

VARBINARY is a variable-length binary string column. It's similar to VARCHAR, but it stores binary data, not text.

Note:

- VARBINARY stores binary strings, not character strings.
- Comparisons are based on numeric byte values, not text collation.
- You can index VARBINARY columns.
- You can assign default values but must be valid binary literals.

varbinary column

col_name VARBINARY

- CREATE TABLE login (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  userName VARCHAR(40),
  password VARBINARY(40) INVISIBLE
) ;

- INSERT INTO login(userName, password) VALUES('ram', 'ram@123');
- INSERT INTO login(userName, password) VALUES('pankaj', 'pankaj');
- INSERT INTO login(userName, password) VALUES('rajan', 'rajan');
- INSERT INTO login(userName, password) VALUES('ninad', 'ninad');
- INSERT INTO login(userName, password) VALUES('omkar', 'omkar');

- SELECT * FROM login;
- SELECT username, CAST(password as CHAR) FROM login;

- CREATE TABLE temp(
  c1 INT PRIMARY KEY,
  c2 VARBINARY(4) DEFAULT 0x89504E47);

a1 INT(10) zerofill	0	0	0	0	0	0	1	5	0	0	length(ID1) -> 10
a2 INT(10)							**3**	**6**	**7**	**5**	length(ID2) -> 4

zerofill value

When you select a column with the type ZEROFILL it pads the displayed value of the field with zeros up to the display width specified in the column definition.

Note:

- ZEROFILL attribute is use for numeric datatype.
- If you specify ZEROFILL for a column, MySQL automatically adds the ***UNSIGNED*** attribute to the column.

zerofill column

col_name data_type ZEROFILL

```
CREATE TABLE employee (
  ID INT ZEROFILL AUTO_INCREMENT PRIMARY KEY,
  firstName VARCHAR(40),
  salary INT,
  commission INT,
  total INT DEFAULT(salary + commission)
) ;
```

- INSERT INTO employee VALUES(NULL, 'ram', 4700, NULL, default);
- INSERT INTO employee VALUES(0, 'pankaj', 3400, 400 , default);
- INSERT INTO employee VALUES(100, 'rajan', 3200, NULL , default);
- INSERT INTO employee VALUES(NULL, 'ninad', 2600, 0, default);
- INSERT INTO employee VALUES(0, 'omkar', 4500, 300, default);
- INSERT INTO employee VALUES (-200, 'rahul', 3000, 300 , default);

- SELECT * FROM employee;
- SELECT ID, LENGTH(ID), salary, LENGTH(salary) FROM employee;

zerofill column

```
CREATE TABLE account (
    accountNumber INT ZEROFILL PRIMARY KEY,
    balance FLOAT,
    openingBalance FLOAT,
    accountName VARCHAR(45),
    custonerID INT,
    openingDate DATE
);
```

Field	Type	Null	Key	Default	Extra
accountNumber	int(10) unsigned zerofill	NO	PRI	NULL	
balance	float	YES		NULL	
openingBalance	float	YES		NULL	
accountName	varchar(45)	YES		NULL	
custonerID	int	YES		NULL	
openingDate	date	YES		NULL	

int

Note:

- If you specify ZEROFILL for a numeric column, MySQL automatically adds the ***UNSIGNED*** attribute to the column.

drop table

Remember:

- DROP and TRUNCATE are DDL commands, whereas DELETE is a DML command.
- DELETE operations can be rolled back (undone), while DROP and TRUNCATE operations cannot be rolled back (DDL statements are auto committed).
- Dropping a TABLE also drops any TRIGGERS for the table.
- Dropping a TABLE also drops any INDEX for the table.
- Dropping a TABLE will not drops any VIEW for the table.
- If you try to drop a PARENT/MASTER TABLE, it will not get dropped.

drop table

DROP [TEMPORARY] TABLE [IF EXISTS] tbl_name [, tbl_name] ...

Note:

- All table data and the table definition are removed/dropped.
- If it is desired to delete only the records but to leave the table definition for future use, then the ***DELETE*** command should be used instead of ***DROP TABLE***.

- DROP login;
- DROP TABLE users;
- DROP TABLE login, users;

continue with SELECT statement…

```
SELECT what_to_select
FROM which_table
WHERE conditions_to_satisfy;
```

The asterisk symbol “ * ” can be used in the SELECT clause to denote “all attributes.”

SELECT CLAUSE

The **SELECT** statement retrieves or extracts data from tables in the database.

- You can use one or more tables separated by comma to extract data.
- You can fetch one or more fields/columns in a single SELECT command.
- You can specify star (*) in place of fields. In this case, SELECT will return all the fields.
- SELECT can also be used to retrieve rows computed without reference to any table e.g. SELECT 1 + 2;

Capabilities of SELECT Statement

1. SELECTION
2. PROJECTION
3. JOINING

Capabilities of SELECT Statement

➢ SELECTION

Selection capability in SQL is to choose the rows in a table that you want to return by a query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ PROJECTION

Projection capability in SQL to choose the columns in a table that you want to return by your query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ JOINING

Join capability in SQL to bring together data that is stored in different tables by creating a link between them.

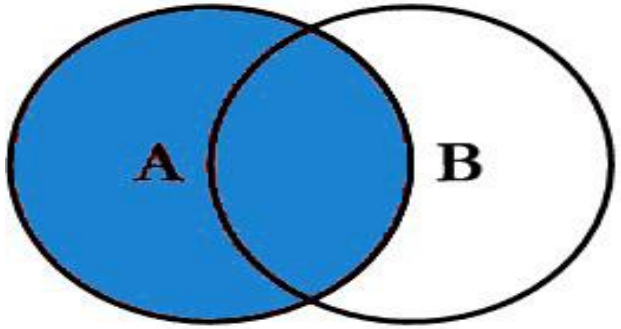
R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	20
2	Janhavi	Sales	1994-12-20	10
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	20
5	Ketan	Sales	1994-01-01	30

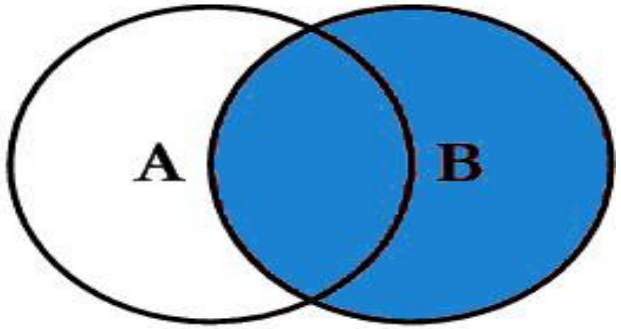
S

DEPTNO	DNAME	LOC
10	HRD	PUNE
20	SALES	BARODA
40	PURCHASE	SURAT

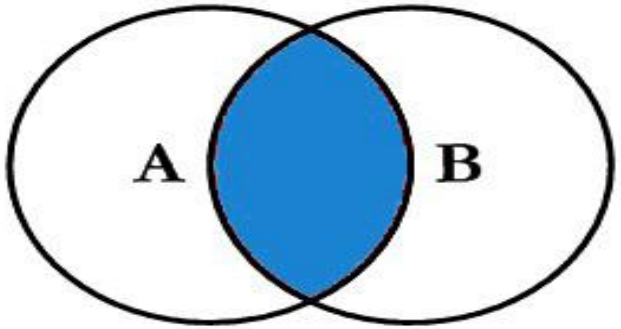
SQL JOINS



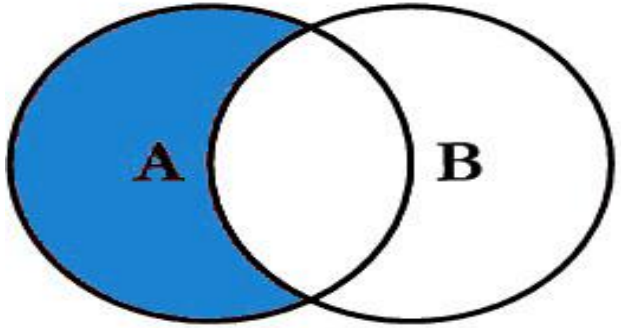

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



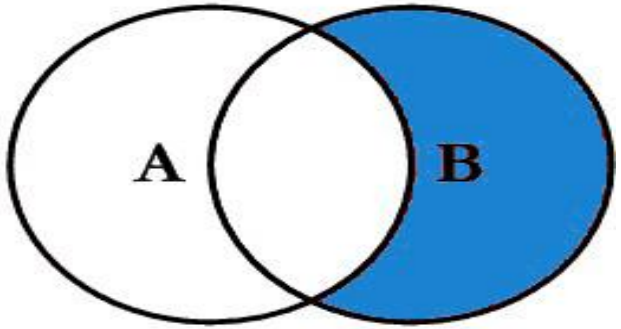

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



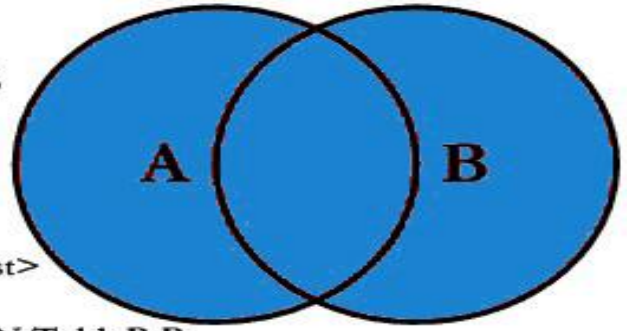

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



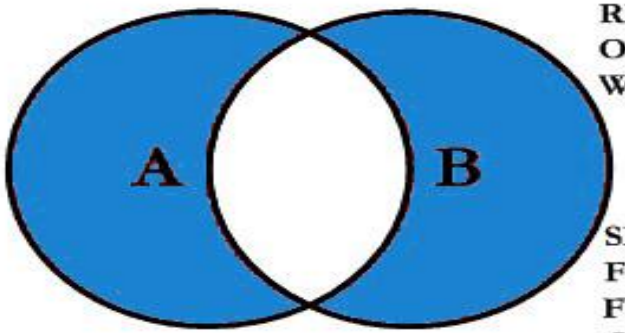

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

SELECTION Process

SELECT * FROM <table_references>

selection-list | field-list | column-list

When you write a SELECT query, the list of columns or expressions you mention after the SELECT keyword is called the SELECT-LIST (also referred to as FIELD-LIST).

Remember:

- Here, " * " is known as metacharacter (all columns)

PROJECTION Process

SELECT column-list FROM <table_references>

selection-list | field-list | column-list

Remember:

- Position of columns in SELECT statement will determine the position of columns in the output (as per user requirements)

anti-pattern in select statement

Using SELECT *** (SELECT all columns)

SELECT * FROM <table_references>

selection-list | field-list | column-list

Anti-pattern

- Here, SELECT * FROM employees;

Why it's bad:

- Fetches unnecessary columns → wastes bandwidth and memory.
- Breaks code if schema changes (e.g., new columns added).

PROJECTION Process

SELECT column-list FROM <table_references>

selection-list | field-list | column-list

Better approach

- Here, SELECT emp_id, emp_name, salary FROM employees;

- SELECT 'HELLO' ' WORLD';
- SELECT 'HELLO' AS 'WORLD';
- SELECT ename `EmployeeName` FROM emp;
- SELECT ename AS `EmployeeName` FROM emp;

column - alias

A programmer can use an alias to temporarily assign another name to a **column** or **table** for the duration of a ***SELECT*** query**.**

In the selection-list, a quoted column alias can be specified using identifier **(** ` **)** or string quote **(** ' or " **)** characters.

Note:

- Assigning an alias_name does not actually rename the column or table.
- You cannot use alias in an expression.

select statement - alias

SELECT A_1 [[AS] *alias_name*], A_2 [[AS] *alias_name*], . . ., A_N FROM *r* [[AS] *alias_name*]

column-name as new-name

table-name as new-name

Remember:

- A select_expr can be given an alias using **AS alias_name**. The alias is used as the expression's column name and can be used in **GROUP BY, HAVING,** or **ORDER BY** clauses.
- The **AS** keyword is optional when aliasing a select_expr with an identifier.
- Standard SQL **disallows** references to column aliases in a ***WHERE*** clause.
- A table reference can be aliased using **tbl_name alias_name** or **tbl_name AS alias_name**
- If the column alias contains spaces, **put it in quotes**.
- Alias name is **max 256 characters**.

- SELECT empno AS *EmployeeID*, ename *EmployeeName* FROM emp;
- SELECT ID AS *'Employee ID'*, ename *"Employee Name"* FROM emp;
- SELECT * FROM emp *employee*;

comparison functions and operator

Comparison operations result in a value of 1 **(TRUE),** 0 **(FALSE)**, or **NULL**.

assignment _operator

= (assignment), :=

- The value on the right hand side may be a literal value, another variable storing a value, or any legal expression that yields a scalar value, including the result of a query (provided that this value is a scalar value). You can perform multiple assignments in the same SET statement. You can perform multiple assignments in the same statement.
- Unlike **=**, the **:=** operator is never interpreted as a comparison operator. This means you can use := in any valid SQL statement (not just in SET statements) to assign a value to a variable.

comparison functions and operator

1. ***arithmetic_operators:***
   * | / | DIV | % |MOD | - | +
2. ***comparison_operator:***
   = | <=> | >= | > | <= | < | <> | !=
3. ***boolean_ predicate:***
   IS [NOT] *NULL* | IS [BOOLEAN]
   | *expr* <=> *null*
4. ***predicate*:**
   expr [NOT] LIKE *expr* [ESCAPE *char*]
   | *expr* [NOT] IN (*expr1, expr2, . . .*)
   | *expr* [NOT] IN (*subquery*)
   | *expr* [NOT] BETWEEN *expr1* AND *expr2*
5. ***logical_operators***
   { AND | && } | { OR | || }
6. ***assignment _operator***
   = (assignment), :=

operand meaning: the quantity on which an operation is to be done.

e.g.

1. *operand1 * operand2*
2. *operand1 = operand2*
3. *operand* IS [NOT] *NULL*
4. *operand* [NOT] LIKE 'pattern'
5. *expr* AND *expr*
6. *Operand := 1001*

- SELECT 23 DIV 6 ; #3
- SELECT 23 / 6 ; #3 .8333

Note:

- AND has higher precedence than OR.